

Corso Blazor Moderno (.NET 10)

Giorno 2: Advanced Patterns, Sicurezza e PWA

Agenda Giorno 2

- **Recap Giorno 1:** Riflessioni su Componenti e Render Modes.
- **Modulo 9:** Interoperabilità con Javascript (JS Interop).
- **Modulo 10:** Gestione dello Stato e Persistenza.
- **Modulo 11:** Sicurezza, Identity e Autorizzazione.
- **Modulo 12:** Progressive Web Apps (PWA) con Blazor.
- **Laboratorio Finale:** Integrazione e Deploy.

Modulo 9: Interoperabilità con Javascript

Quando C# ha bisogno di un "aiutante"

9.1. Perché usare JS Interop?

Nonostante Blazor copra il 95% dei casi d'uso, JS serve per:

- **Librerie Esterne:** Chart.js, Leaflet, editor di testo avanzati.
- **Browser API:** Geolocalizzazione, Bluetooth, Web Crypto (non ancora tutte mappate).
- **Performance Grafiche:** Manipolazione diretta di Canvas 2D/3D (WebGL).

9.2. Iniezione e Chiamata

Si utilizza l'interfaccia `IJSRuntime`.

```
@inject IJSRuntime JS

<button @onclick="ShowAlert">Esegui JS</button>

@code {
    private async Task ShowAlert() {
        await JS.InvokeVoidAsync("alert", "Messaggio inviato da C#!");
    }
}
```

9.3. JS Isolation (Moduli)

In .NET 10, la best practice è caricare JS solo quando serve tramite i Moduli ES6:

```
private IJSObjectReference? _module;

protected override async Task OnAfterRenderAsync(bool firstRender) {
    if (firstRender) {
        _module = await JS.InvokeAsync<IJSObjectReference>(
            "import", "./scripts/mio-modulo.js");
    }
}
```

- Vantaggio: Nessun conflitto tra nomi globali e caricamento "lazy".

Esercitazione modulo 9: Interoperabilità con JS (1)

Obiettivo: Creare una pagina di dettaglio di un dispositivo che contiene un grafico di Chart.Js

Step 1: Creiamo la pagina **DeviceDetails.razor**

```
@page "/device/{Id:int}"
@using Microsoft.EntityFrameworkCore
@using Microsoft.JSInterop
@implements IAsyncDisposable
@inject SmartDbContext Db
@inject IJSRuntime JS
@inject NavigationManager Nav
@rendermode InteractiveAuto
```

```

<PageTitle>Dettaglio - @(device?.Name ?? "Caricamento")</PageTitle>

@if (device == null)
{
    <div class="text-center mt-5">
        <div class="spinner-border text-primary" role="status"></div>
        <p>Recupero informazioni dal database...</p>
    </div>
}
else
{
    <nav aria-label="breadcrumb">
        <ol class="breadcrumb">
            <li class="breadcrumb-item"><a href="/" @onclick:preventDefault @onclick='() => Nav.NavigateTo("/")'>Dashboard</a></li>
            <li class="breadcrumb-item active">@device.Name</li>
        </ol>
    </nav>
    <div class="row">
        <div class="col-md-4">
            <div class="card shadow-sm mb-4">
                <div class="card-header bg-primary text-white">
                    <h5 class="mb-0">Informazioni Dispositivo</h5>
                </div>
                <div class="card-body">
                    <p><strong>Nome:</strong> @device.Name</p>
                    <p><strong>Stato:</strong>
                        <span class="badge @(device.IsOn ? "bg-success" : "bg-danger")">
                            @(device.IsOn ? "Attivo" : "Spento")
                        </span>
                    </p>
                </div>
            </div>
        </div>
    </div>
}

```



```

        <p><strong>Valore Corrente:</strong> @device.Value °C</p>
        <hr />
        <button class="btn btn-outline-primary w-100" @onclick='() => Nav.NavigateTo("/")'>
            Torna alla Dashboard
        </button>
    </div>
</div>
</div>
<!-- Colonna Grafico Storico -->
<div class="col-md-8">
    <div class="card shadow-sm">
        <div class="card-body">
            <h5 class="card-title">Storico ultime 24 ore</h5>
            <div style="height: 350px;">
                <canvas id="deviceDetailChart"></canvas>
            </div>
        </div>
    </div>
</div>
</div>
}

```

```

@code {
    [Parameter] public int Id { get; set; }
    private IoTDevice? device;
    private IJSObjectReference? _module;

    // FASE 1: Recupero dati dal DB (C#)
    protected override async Task OnInitializedAsync()
    {
        device = await Db.Devices.FirstOrDefaultAsync(d => d.Id == Id);

        if (device == null)
        {
            // Se l'ID non esiste, riportiamo l'utente in home (o una 404)
            Nav.NavigateTo("/");
        }
    }
}

```

```

// FASE 2: Inizializzazione Grafico (JS Interop)
protected override async Task OnAfterRenderAsync(bool firstRender)
{
    if (firstRender && device != null)
    {
        // Simulazione 24 punti storici
        var labels = Enumerable.Range(0, 24)
            .Select(i => DateTime.Now.AddHours(-23 + i).ToString("HH:00"))
            .ToArray();

        var random = new Random();
        var dataPoints = Enumerable.Range(0, 24)
            .Select(_ => Math.Round(device.Value + (random.NextDouble() * 2 - 1), 1))
            .ToArray();

        // Import modulo dalla RCL
        _module = await JS.InvokeAsync<IJSObjectReference>("import",
            "./_content/SmartHub.UI/js/chartModule.js");

        // Render grafico
        await _module.InvokeVoidAsync("renderLineChart", "deviceDetailChart", labels, dataPoints);
    }
}

public async ValueTask DisposeAsync()
{
    if (_module is not null) await _module.DisposeAsync();
}
}

```

Esercitazione modulo 9: Interoperabilità con JS (2)

Step 2: Collegare la home alla pagina di dettaglio del dispositivo (verifica che sia la pagina di dettaglio sia il grafico vengono caricati correttamente)

```
<div class="card h-100 shadow-sm">
  <div class="card-body text-center">
    <h5 class="card-title">@Device.Name</h5>
    <h2 class="display-6">@Device.Value°</h2>
    <div class="d-grid gap-2 mt-3">
      <button class="btn btn-primary" @onclick="HandleToggle">
        @(Device.IsOn ? "Spegni" : "Accendi")
      </button>
      <!-- Link al dettaglio -->
      <a href="/device/@Device.Id" class="btn btn-outline-secondary">
        Vedi Dettagli
      </a>
    </div>
  </div>
</div>
```

Modulo 10: Gestione dello Stato

Sincronizzazione tra Server e Client

10.1. Il problema dello stato nelle SPA

- Blazor Server: Lo stato vive nella memoria del server (Circuit).
- Blazor WASM: Lo stato vive nella memoria del browser.
- Refresh della pagina: Distrugge lo stato in memoria.

10.2. Strategie di Persistenza (1)

1. In-Memory: Servizi "Scoped" o "Singleton".
2. Component State: Passaggio di parametri.
3. Browser Storage: LocalStorage o SessionStorage (via JS Interop).
4. PersistentComponentState: Servizio nativo per passare dati dal server al client durante la Hydration (.NET 8/10).

10.3. Strategie di Persistenza (2)

State Container Pattern

Quinta e ultima strategia di persistenza

Creare una classe centralizzata registrata come servizio:

```
public class CartState {  
    public int Count { get; private set; }  
    public event Action? OnChange;  
  
    public void Add() {  
        Count++;  
        NotifyStateChanged();  
    }  
    private void NotifyStateChanged() => OnChange?.Invoke();  
}
```


Modulo 11: Sicurezza e Identity

Proteggere l'applicazione in .NET 10

11.1 Blazor Identity UI

In .NET 10, il sistema di autenticazione è completamente Blazor-Native:

- Niente più "Razor Pages" esterne per Login/Register.
- Gestione unificata di Cookie (Server-side) e Token (Client-side).
- Componenti pronti all'uso per MFA (Multi-Factor Authentication).

11.2 Autorizzazione Dichiarativa

```
<AuthorizeView Roles="Admin">
  <Authorized>
    <button>Cancella Record</button>
  </Authorized>
  <NotAuthorized>
    <p>Solo gli amministratori possono vedere questo pulsante.</p>
  </NotAuthorized>
</AuthorizeView>
```

- Protezione delle rotte: @attribute [Authorize] in cima alla pagina.

Esercitazione modulo 11: Aggiungere e configurare Identity (1)

Step 1: Aggiorna il database, modificando SmartDbContext in questo modo

```
// Cambia l'ereditarietà da DbContext a IdentityDbContext
public class SmartDbContext : IdentityDbContext<IdentityUser>
{
    public SmartDbContext(DbContextOptions<SmartDbContext> options) : base(options) { }

    public DbSet<IoTDevice> Devices => Set<IoTDevice>();

    // ... OnModelCreating rimane uguale
}
```

Step 2: Aggiunge la migrazione e aggiorna il DB

```
dotnet ef migrations add AddIdentityTables --project SmartHub.Data --startup-project SmartHub.App
dotnet ef database update --project SmartHub.Data --startup-project SmartHub.App
```

Esercitazione modulo 11: Aggiungere e configurare Identity (2)

Step 3: Configurazione nel program.cs

```
// Program.cs (Server)

builder.Services.AddCascadingAuthenticationState(); // Fondamentale per AuthorizeView
builder.Services.AddScoped<IdentityUserAccessor>(); // Helper per recuperare l'utente corrente
builder.Services.AddScoped<IdentityRedirectManager>(); // Gestisce i redirect dopo login
// Configura Identity
builder.Services.AddIdentityCore<IdentityUser>(options => options.SignIn.RequireConfirmedAccount = false)
    .AddEntityFrameworkStores<SmartDbContext>()
    .AddSignInManager()
    .AddDefaultTokenProviders();
builder.Services.AddAuthentication(options =>
{
    options.DefaultScheme = IdentityConstants.ApplicationScheme;
    options.DefaultSignInScheme = IdentityConstants.ExternalScheme;
})
    .AddIdentityCookies();
// ... dopo builder.Build()
app.MapAdditionalIdentityEndpoints(); // Mappa le rotte di Login/Logout
```

Esercitazione modulo 11: Aggiungere e configurare Identity (3)

Step 3: Protezione della pagina di aggiunta dispositivo

```
@attribute [Authorize]
```

Step 4: Rendering condizionale nel layout

```
<div class="top-row px-4">
  <AuthorizeView>
    <Authorized>
      <span class="me-3">Ciao, @context.User.Identity?.Name!</span>
      <form action="Account/Logout" method="post">
        <AntiforgeryToken />
        <button type="submit" class="btn btn-link nav-link">Logout</button>
      </form>
    </Authorized>
    <NotAuthorized>
      <a href="Account/Register">Registrati</a>
      <a href="Account/Login" class="ms-3">Login</a>
    </NotAuthorized>
  </AuthorizeView>
</div>
```

Esercitazione modulo 11: Aggiungere e configurare Identity (4)

Step 5: Autorizzazione nel dettaglio con ruoli

```
@* In DeviceDetails.razor *@  
  
<AuthorizeView Roles="Admin">  
    <div class="alert alert-warning">  
        <strong>Area Tecnica:</strong> Il firmware del dispositivo è v10.4.2  
    </div>  
</AuthorizeView>
```


Esercitazione modulo 11: Concetti chiave

Authentication vs Authorization

- **Authentication:** "Chi sei?" (Login).
- **Authorization:** "Cosa puoi fare?" (Ruoli/Policy).

AuthorizeView

Il componente magico che nasconde/mostra pezzi di interfaccia in base allo stato dell'utente senza scrivere `if` complessi.

CascadingAuthenticationState

In .NET 10, lo stato dell'utente "scorre" attraverso tutti i componenti. Se l'utente fa il login sul server, il client WASM viene aggiornato tramite la **Hydration**.

Modulo 12: PWA con Blazor

Installare il web sul desktop

12.1 Caratteristiche PWA

- Installabilità: L'app appare nel menu Start o sulla Home del telefono.
- Service Worker: Script JS che gira in background per gestire la cache.
- Manifest.json: Definisce icone, colori e nome app.

12.2 Supporto Offline in .NET 10

- Caching Strategico: Scarica le DLL .NET e le risorse statiche per l'uso offline.
- Background Sync: Possibilità di inviare form salvati localmente non appena torna la connessione (novità migliorata in .NET 10).

Esercitazione modulo 12: Trasformiamo la nostra app in una PWA (1)

Step 1: Aggiungere il manifest.json (nella wwwroot dell'app)

```
{
  "name": "SmartHub IoT",
  "short_name": "SmartHub",
  "start_url": "./",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#512bd4",
  "icons": [
    { "src": "icon-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

Esercitazione modulo 12: Trasformiamo la nostra app in una PWA (2)

Step 2: Aggiungere il service-worker.json (nella wwwroot dell'app)

In .NET 10, il compilatore genera automaticamente una lista di asset (comprese le DLL della nostra RCL) che il worker metterà in cache.

Step 3: Rilevare lo stato online/offline (JSInterop)

```
export function registerConnectionHandler(dotNetHelper) {  
    window.addEventListener('online', () => dotNetHelper.invokeMethodAsync('SetOnlineStatus', true));  
    window.addEventListener('offline', () => dotNetHelper.invokeMethodAsync('SetOnlineStatus', false));  
    return window.navigator.onLine;  
}
```

Esercitazione modulo 12: Trasformiamo la nostra app in una PWA (3)

Step 4: Rilevare lo stato online/offline (C#)

Serve un servizio .NET per informare WASM riguardo lo stato della rete

```
public class NetworkService : IDisposable {  
    private readonly IJSRuntime _js;  
    private DotNetObjectReference<NetworkService>? _objRef;  
    public bool IsOnline { get; private set; } = true;  
    public event Action<bool>? OnStatusChanged;  
  
    public NetworkService(IJSRuntime js) => _js = js;
```

```

public async Task InitializeAsync() {
    _objRef = DotNetObjectReference.Create(this);
    var module = await _js.InvokeAsync<IJSObjectReference>("import", "./_content/SmartHub.UI/js/network.js");
    IsOnline = await module.InvokeAsync<bool>("registerConnectionHandler", _objRef);
}

[JSInvokable]
public void SetOnlineStatus(bool isOnline) {
    IsOnline = isOnline;
    OnStatusChanged?.Invoke(isOnline);
}

public void Dispose() => _objRef?.Dispose();
}

```


Esercitazione modulo 12: Trasformiamo la nostra app in una PWA (4)

Step 5: Feedback visivo nel Layout

```
@inject NetworkService Network
@implements IDisposable

<div class="main">
  @if (!isOnline)
  {
    <div class="alert alert-warning text-center mb-0" style="border-radius:0;">
      ⚠ Modalità Offline: I comandi ai dispositivi sono disabilitati.
    </div>
  }
  <div class="content px-4">
    @Body
  </div>
</div>
```

```
@code {  
    private bool isOnline = true;  
  
    protected override async Task OnAfterRenderAsync(bool firstRender) {  
        if (firstRender) {  
            Network.OnStatusChanged += HandleStatusChange;  
            await Network.InitializeAsync();  
            isOnline = Network.IsOnline;  
            StateHasChanged();  
        }  
    }  
  
    private void HandleStatusChange(bool status) {  
        isOnline = status;  
        InvokeAsync(StateHasChanged);  
    }  
  
    public void Dispose() => Network.OnStatusChanged -= HandleStatusChange;  
}
```

Conclusioni e Q&A

Grazie per la partecipazione!

Daniele Gastaldo

-  daniele.gastaldo@reteinformatica.com